

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track • Windows • Terminal-first

MONTH 5 — LAB GUIDE

Build a Sentiment Analyzer Agent

An agent that reads text and tells you the mood behind it — and how sure it is.

Builds on Months 1–4. This month you build your very first classifier — a program that makes a judgement.

What you will build this month

- An agent that reads a sentence and decides if it's positive, negative, or neutral.
- It reports a confidence level — how sure it is about its answer.
- It **LEARNS** the mood of words from example data, instead of you hard-coding them.
- It is tested honestly on real reviews and shipped to GitHub as v1.0.

New this month: your first classifier & a little probability

- A **classifier** is a program that sorts things into categories — here, sorting text into moods.
- You'll meet **probability** (how likely / how sure) and the idea behind **Bayes** — gently, through building.
- You'll also meet **training data**: teaching a program by showing it examples instead of writing rules by hand.

Month 5 at a glance

Until now your agents followed rules you wrote or measured similarity. This month your agent makes a judgement — it reads a piece of text and decides its mood. That's called classification, and it's one of the most useful things in all of AI. You'll start with simple word-counting, add a sense of confidence using probability, and finish by letting the agent LEARN from examples rather than being told every rule.

Week	Focus	What you ship
Week 1	Words as signals & counting	A bot that scores text by word cues (v0.1)
Week 2	Probability: how sure are we?	A bot that gives a confidence level (v0.2)
Week 3	Learning from examples (training data)	A bot that learns word scores from data (v0.3)
Week 4	Measure accuracy honestly & ship	A tested sentiment agent on real reviews (v1.0)

The journey this month: Week 1 you hard-code which words are positive. Week 2 you turn counts into a confidence score. Week 3 is the magic — the agent learns word moods from data by itself. Week 4 you check, honestly, how good it really is.

Week 1 — Words as signals & counting

Goal of the week: the simplest possible mood detector — count positive and negative words in a sentence and decide which wins.

Day 1 — Positive and negative word lists

The simplest idea: some words feel positive (good, love, great), some negative (bad, hate, awful). Start by listing them.

Task 1.1 — Make the word lists

1. Create a folder `sentiment`, open it in VS Code, and make `day1.py`:

```
positive_words = ["good", "great", "love", "happy", "excellent", "best"]
negative_words = ["bad", "hate", "awful", "terrible", "worst", "sad"]

print("Positive words I know:", positive_words)
```

2. Save and run to confirm your lists print.

 **Checkpoint:** You have two lists of mood words — the raw signals your agent will look for.

Day 2 — Count the mood words in a sentence

Split a sentence into words and count how many are positive vs negative. This reuses lists and loops from Month 2.

Task 2.1 — Count positives and negatives

3. New file `day2.py` (keep the word lists at the top):

```
text = "the food was good but the service was bad"
words = text.split()

pos_count = 0
neg_count = 0
for word in words:
    if word in positive_words:
        pos_count = pos_count + 1
    if word in negative_words:
        neg_count = neg_count + 1

print("Positive words:", pos_count)
print("Negative words:", neg_count)
```

✓ What you should see

```
Positive words: 1
Negative words: 1
```

`.split()` breaks a sentence into a list of individual words. Then you loop and count matches against each list — the same counting pattern from your quiz score.

 **Checkpoint:** You can count positive and negative words inside any sentence.

Day 3 — Make the decision

Whichever count is higher decides the mood — with a tie meaning neutral.

Task 3.1 — Decide the sentiment

4. Add below the counts:

```
if pos_count > neg_count:
    print("Sentiment: POSITIVE 😊")
elif neg_count > pos_count:
    print("Sentiment: NEGATIVE 😞")
else:
    print("Sentiment: NEUTRAL 😐")
```

5. Save and run. Try changing the sentence to more positive or more negative words.

✓ What you should see

Sentiment: NEUTRAL 😐

🚩 **Checkpoint:** You built a working (if simple) mood detector. That's a classifier!

Day 4 — Wrap it in a function & clean the text

Task 4.1 — analyze(text)

6. Turn it into a reusable function (Month 1 skill), and lowercase the text so “Good” matches “good” (Month 2 trick):

```
def analyze(text):
    text = text.lower()
    words = text.split()
    pos = sum(1 for w in words if w in positive_words)
    neg = sum(1 for w in words if w in negative_words)
    if pos > neg:
        return "positive"
    elif neg > pos:
        return "negative"
    return "neutral"

print(analyze("I LOVE this, it is great"))
```

✓ What you should see

positive

The line `sum(1 for w in words if w in positive_words)` is a compact way to count matches — it adds 1 for each matching word. A neat shortcut for the loop you wrote on Day 2.

🚩 **Checkpoint:** Your detector is a clean, reusable function that ignores capitalisation.

Days 5–7 — Interactive agent & ship v0.1

- Day 5: Add a chat loop (Month 1) so the user can type sentences and get the mood, until they type `quit`.

- Day 6: Grow your word lists to 20+ each. Notice accuracy improves as the lists grow — a hint that MORE knowledge helps.
- Day 7: Build v0.1 of `sentiment.py`: word lists, the `analyze()` function, and an interactive loop. Test it on your own sentences.

 **Checkpoint:**  Week 1 done. You have a working mood detector. Next week: how SURE is it?

Week 2 — Probability: how sure are we?

Goal of the week: a real agent doesn't just say "positive" — it says "80% positive." This week you turn counts into a confidence score, meeting probability along the way.

Day 8 — What is probability?

Probability is just "how likely," written as a number from 0 (never) to 1 (certain), often shown as a percentage.

Task 8.1 — Feel probability with a simple example

7. New file `day8.py`:

```
pos = 3    # positive words found
neg = 1    # negative words found
total = pos + neg

prob_positive = pos / total
print("Chance it is positive:", prob_positive)
print("As a percentage:", prob_positive * 100, "%")
```

✓ What you should see

Chance it is positive: 0.75

As a percentage: 75.0 %

If 3 of 4 mood-words are positive, it's 3 out of 4 = 0.75 = 75% likely positive. Probability is just a part divided by the whole — a fraction between 0 and 1.

 **Checkpoint:** You understand probability as a 0-to-1 fraction, and can compute it from counts.

Day 9 — A confidence score for your agent

Task 9.1 — Return confidence, not just a label

8. Upgrade `analyze()` to also return how sure it is:

```
def analyze(text):
    text = text.lower()
    words = text.split()
    pos = sum(1 for w in words if w in positive_words)
    neg = sum(1 for w in words if w in negative_words)
    total = pos + neg
    if total == 0:
        return "neutral", 0.0
    if pos > neg:
        return "positive", pos / total
    elif neg > pos:
        return "negative", neg / total
    return "neutral", 0.5
```

9. Test it:

```
label, confidence = analyze("great great good but bad")
```

```
print(label, "with confidence", round(confidence*100), "%")
```

✓ **What you should see**

positive with confidence 75 %

Watch the `if total == 0` line — without it, dividing by zero (a sentence with no mood words) would crash your program.

🚩 **Checkpoint:** Your agent now reports how confident it is, not just a bare label.

Day 10 — The idea behind Bayes (gently)

Bayes thinking = start with a belief, then update it as you see evidence. Each mood word is a clue that nudges your belief toward positive or negative.

Task 10.1 — Watch belief update word by word

10. Print the running counts as you scan a sentence, to SEE the belief shift:

```
text = "good good bad"
pos, neg = 0, 0
for word in text.split():
    if word in positive_words: pos += 1
    if word in negative_words: neg += 1
    print(f"after '{word}': pos={pos} neg={neg}")
```

✓ **What you should see**

```
after 'good': pos=1 neg=0
after 'good': pos=2 neg=0
after 'bad': pos=2 neg=1
```

This is Bayes in spirit: each new word is evidence that updates your belief. You start unsure, and every clue shifts the balance. Real Bayesian classifiers do exactly this with probabilities.

🚩 **Checkpoint:** You understand the core Bayesian idea: evidence updates belief, one clue at a time.

Days 11-14 — Handle negation & ship v0.2

- Day 11: Handle “not”: “not good” should count as negative. Detect when **not** appears just before a positive word and flip it.
- Day 12: Show the confidence as a friendly bar or percentage in the agent's replies.
- Day 13: Test tricky sentences (mixed, sarcastic, no mood words) and note where it struggles — that's honest engineering.
- Day 14: Ship v0.2 — a mood agent that reports confidence and handles negation.

🚩 **Checkpoint:** End of Week 2: your agent gives a mood AND a confidence level, and handles “not”.

Week 3 — Learning from examples (training data)

Goal of the week: so far YOU decided which words are positive. This week the agent LEARNS word moods from example data on its own — your first real taste of machine learning.

Day 15 — What is training data?

Training data = labelled examples. For sentiment, that's sentences each tagged positive or negative. The agent studies them to learn which words go with which mood.

Task 15.1 — Create a tiny training set

11. New file `day15.py`:

```
training = [
    ("I love this movie", "positive"),
    ("what a great film", "positive"),
    ("this was awful", "negative"),
    ("I hate it so bad", "negative"),
]

for text, label in training:
    print(label, "->", text)
```

Each item is a (sentence, correct label) pair. This is exactly the “list of pairs” shape you used before — now it's teaching material for the agent.

 **Checkpoint:** You have labelled training data: examples with their correct moods.

Day 16 — Learn which words are positive

The agent counts: for each word, how often does it appear in positive vs negative examples? That count IS the learning.

Task 16.1 — Build word-mood counts from data

12. Count each word under each label using a dictionary (Month 2):

```
pos_counts = {}
neg_counts = {}
for text, label in training:
    for word in text.lower().split():
        if label == "positive":
            pos_counts[word] = pos_counts.get(word, 0) + 1
        else:
            neg_counts[word] = neg_counts.get(word, 0) + 1

print("love appears in positive:", pos_counts.get("love", 0))
print("awful appears in negative:", neg_counts.get("awful", 0))
```

✓ **What you should see**

```
love appears in positive: 1
awful appears in negative: 1
```

`.get(word, 0)` reads a count, or gives 0 if the word isn't there yet — a safe way to tally into a dictionary without crashing. The agent just LEARNED word moods from examples, no hand-written lists!

 **Checkpoint:** Your agent learns which words lean positive or negative directly from data.

Day 17 — Classify new text using what was learned

- For a new sentence, add up each word's learned positive and negative counts, and pick the higher total.
- Test it on a sentence the agent has never seen — this is the real test of learning.

Days 18-21 — More data, save the model & ship v0.3

- Day 18: Grow your training data to 20+ labelled sentences. Watch predictions improve with more examples.
- Day 19: Save the learned counts to a file (Month 2 file skills) so the agent doesn't re-learn every run — this saved knowledge is called a “model”.
- Day 20: Load a real labelled dataset (CSV of reviews with labels, using Month 3 pandas) and learn from it.
- Day 21: Commit and push to GitHub. This is v0.3.

 **Checkpoint:** Your agent learns word moods from real data and saves what it learned. That's machine learning!

Week 4 — Measure accuracy honestly & ship v1.0

Goal of the week: a model is only trustworthy if you test it fairly. This week you measure how accurate it really is — the honest way — then polish and ship v1.0.

Day 22 — Split data into train and test

The golden rule of ML: never test on the same data you trained on — that's like giving students the exam answers. Keep some examples hidden to test fairly.

Task 22.1 — Hold back a test set

13. Split your labelled data: most for training, a portion kept aside for testing:

```
all_data = training    # your full labelled list
split = int(len(all_data) * 0.8)
train_set = all_data[:split]    # first 80% to learn from
test_set = all_data[split:]    # last 20% kept hidden to test
print("Training on", len(train_set), "| testing on", len(test_set))
```

Learn from train_set only, then check answers against test_set — data the agent never saw. This tells you how it does on NEW text, which is what matters.

 **Checkpoint:** You can split data so you test fairly on unseen examples.

Day 23 — Measure accuracy

Task 23.1 — Count how many it gets right

14. Run the agent on each test sentence and compare to the correct label:

```
correct = 0
for text, true_label in test_set:
    guess = classify(text)    # your learned classifier
    if guess == true_label:
        correct = correct + 1

accuracy = correct / len(test_set)
print("Accuracy:", round(accuracy * 100), "%")
```

✓ **What you should see**

Accuracy: 80 %

Accuracy = how many it got right ÷ how many it tried. A simple, honest score of quality — and it's a probability too (0 to 1)!

 **Checkpoint:** You can measure your agent's real accuracy on unseen data.

Day 24 — See WHERE it goes wrong

- Print the sentences it got wrong. Look for patterns — sarcasm? negation? unknown words?
- Fixing the biggest failure pattern is the fastest way to improve. This is real ML practice.

Day 25 — Make it robust & friendly

- Handle unknown words (never seen in training) gracefully, and empty input without crashing.
- Add a neutral option when confidence is low, so it doesn't force a wrong guess.

Days 26-30 — Final polish & ship v1.0

- Day 26: Test on a bigger real review dataset. Record the accuracy honestly, good or bad.
- Day 27: Write a **README.md** explaining what it does, how it learns, and its measured accuracy.
- Day 28: Record a short demo — type sentences, show mood + confidence, then show the accuracy score.
- Day 29: Final commit and push (**git add .** → **git commit -m "v1.0"** → **git push**).
- Day 30: Show it off. You built an agent that reads mood, learns from data, and knows how good it is.

Month 5 complete

You built your first classifier — a program that makes a judgement — and, more importantly, you taught it to **LEARN** from examples instead of following hand-written rules. You met probability (how sure), the Bayesian idea (evidence updates belief), training data, and honest accuracy testing. These are the true foundations of machine learning. Month 6 takes the final scratch-era step: you'll train a model with gradient descent that you code by hand, fully understanding how a model learns.